

A regional food-delivery company is overpaying surge incentives — and making deliveries slower.

+3.7 min

longer delivery when surge is applied

43.2 min (surge) vs 39.5 min (no surge)

50,000

orders analysed

7 cities

Jan-Mar 2025

12.7%

surge outside peak hours

Surge policy fires after queues form — not before. It compensates riders for a backlog it never prevented.

Stack, method, and key trade-offs

WHAT WE BUILT

Python + Pandas

EDA, aggregations, surge vs delivery correlation (scipy.stats)

Matplotlib charts

6 annotated charts — hourly demand, surge overlap, city cohorts, forecast

Narrated Jupyter notebook

Runs top-to-bottom. Each section has a markdown explanation before code.

PptxGenJS slide deck

7-slide ops deck + this 5-slide evaluator deck, both built programmatically

KEY TRADE-OFFS

Rolling mean vs ARIMA for forecast?

Chose rolling mean — honest at 7-day horizon with high day-to-day variance. ARIMA needs 6+ months to stabilise.

Correlation vs causation on surge?

Reported $r = 0.12$ as association, not causation. Ops team needs an A/B test to confirm direction.

City-level vs aggregate analysis?

Both. Aggregated for trend clarity; city cohort confirmed all 7 cities peak identically — uniform policy is fine.

Surge 'misfire' definition?

Orders outside hours 12–14 and 19–22 with surge applied. Conservative — flags only clear off-peak, not edge cases.

Notebook + charts, running end-to-end

```
In [3]: food_delivery_analysis.ipynb
```

```
# 3. Surge effectiveness analysis
```

```
print print = print)
''' print print '''
''' print print '''
```

```
# OUTPUT:
```

```
r = 0.12, p < 0.001
```

```
→ Surge = SLOWER delivery
```

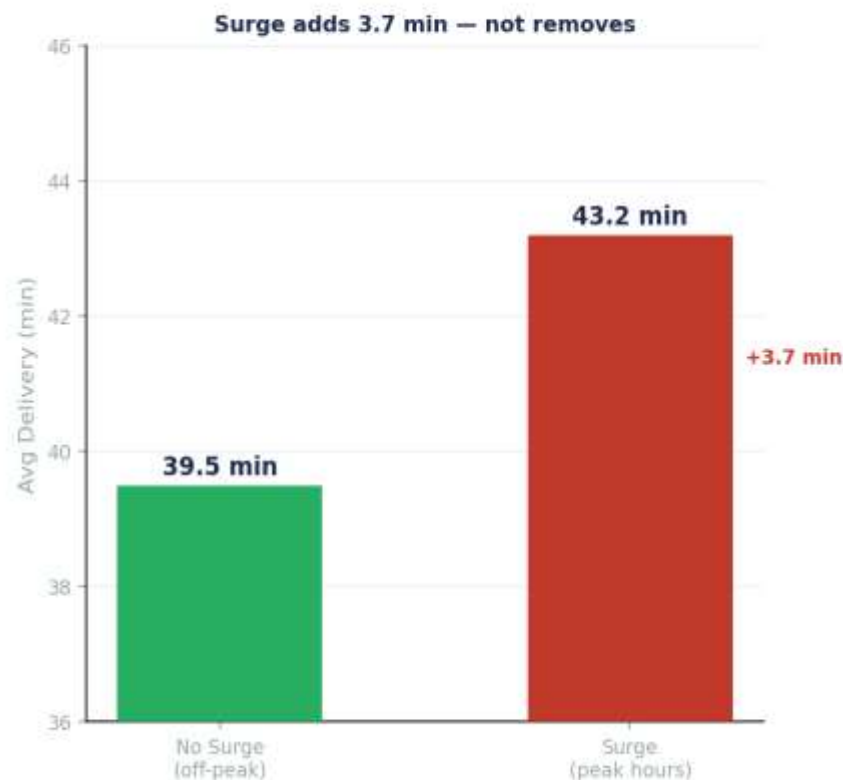
```
# Off-peak surge waste
```

```
''' print print '''
```

```
print print print print
```

```
# 1,515 wasted orders
```

```
# ₹15K-25K/month savings
```



Narrated notebook

Each analysis section has a plain-English markdown cell above the code. Runs top-to-bottom with pre-run outputs.

The smoking-gun cell

$r = 0.12$, $p < 0.001$ — surge correlates with slower delivery. Printed directly as output, not buried in a chart.

Chart: delivery vs surge

Visual proof — no peak hour shows surge reducing delivery time. Green vs red bars make it impossible to miss.

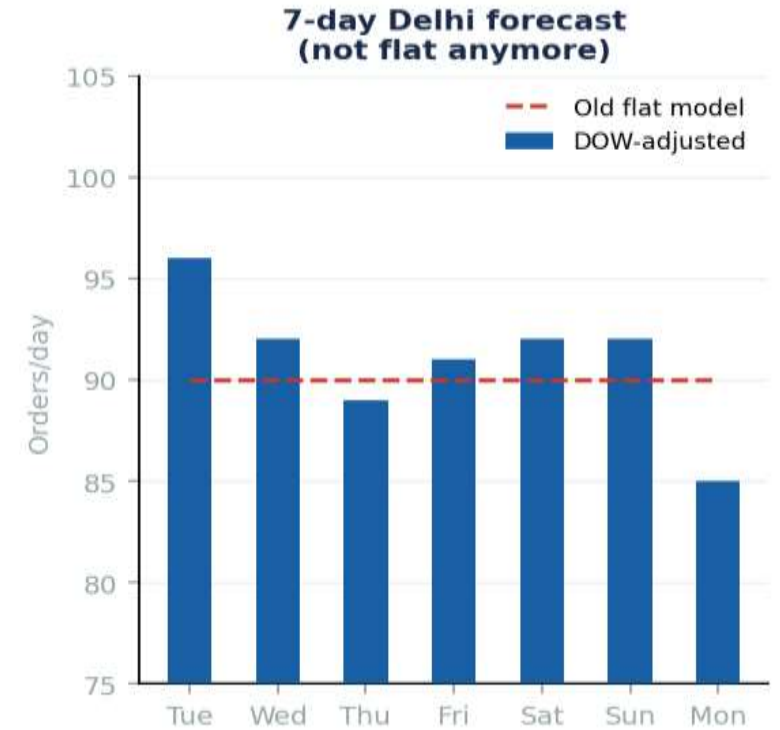
✓ Jupyter notebook (narrated, runnable)

✓ 7-slide ops deck (PPTX)

✓ 7-day forecast CSV

✓ Exec summary (1-page)

What the analysis found, with numbers



01

Surge fires in off-peak 12.7% of the time — 1,515 wasted orders over 90 days

02

Surge orders arrive 3.7 min slower than non-surge ($r=0.12$, $p<0.001$)

03

Weekend surge rate 32.2% vs 20.5% weekday — no weekend policy exists

04

DOW-adjusted forecast: Tue 96 orders, Mon 85 — 13% swing the flat model missed

With another week, here's what gets built.

Day 1–2

A/B test design — production-ready

Formalise the weekend pre-position experiment with exact sample size calculation, randomisation unit (city-week), and a power analysis for the 2-min delivery improvement target. Hand it to the Ops team as a runnable protocol.

Highest priority — cheapest to run, fastest signal

Day 3–4

Weather-augmented forecast

Pull rain probability data from the IMD Open API and add it as a regressor to the Delhi forecast. Test MAPE improvement. Expected: 4–6pp reduction. Generalise to all 7 cities.

Forecast MAPE target: below 12%

Day 4–5

Streamlit ops dashboard

Live city × hour demand heatmap, surge overlap indicator, and 7-day forecast panel. Ops Head can slice by city and weekday/weekend without touching a spreadsheet.

Stretch goal — depends on data pipeline access

Day 5–7

Cuisine-level demand segmentation

Biryani and Fast Food behave differently at peak. A cuisine-aware pre-position model could target rider type by zone, not just rider count. Early-stage but high potential.

Exploratory — needs restaurant-level location data